CS5785 / ORIE5750 / ECE5414 Midterm Project

This midterm project is due on **Saturday, November 7th at 11:59 PM ET**, uploaded to Gradescope (Canvas->Gradescope) and Kaggle. Your submission will have three parts:

1. Submit predictions to the Kaggle competitions for Part 1 and Part 2.
2. A write-up as a single `.pdf` file. Submit this under the `midterm-report` assignment in Gradescope.
3. Source code and data files for all of your experiments (AND figures) in `.ipynb` files (file format for IPython Jupyter Notebook). These files should be placed in a folder titled `midterm` and uploaded to the `midterm-code` assignment in Gradescope. You could use colab notebook template as a starting point.

The write-up should contain a general summary of what you did, how well your solution works, any insights you found, etc. On the cover page, include the class name, and the names of the entire team. In addition, briefly describe the responsibility of each team member. You are responsible for submitting clear, organized answers to the questions.

Please include all relevant information for a question, including text response, equations, figures, graphs, output, etc. If you include graphs, be sure to include the source code that generated them. Please pay attention to Canvas for announcements, policy changes, etc. and Piazza for homework related questions.

IF YOU NEED HELP

There are several strategies available to you.

- Please ask clarification questions on Ed. That way, your solutions will be available to other students in the class.
- You might be able to get some hints by attending the professor and TAs office hours, but the hints are not guaranteed.
- You are allowed to use well known libraries such as `scikit-learn`, `scikit-image`, `numpy`, `scipy`, etc. for this assignment (including implementations of machine learning algorithms), unless we explicitly say that you cannot in a particular question. Any reference or copy of public code repositories should be properly cited in your submission (examples include Github, Wikipedia, Blogs).

PROJECT GUIDELINES

1. **Project Overview** In class, we have worked with text datasets with labels for supervised learning based binary classification, and unsupervised learning techniques. But what about semi-supervised learning tasks? Semi-supervised learning is a machine learning paradigm in which a model is trained on a dataset that contains both labeled and unlabeled examples. In this project, you will investigate semi-supervised learning techniques applied to the multi-label task of predicting five clinical-trial stopping reasons (Safety, Funding, Efficacy, Recruitment, and Other) with partially missing reason labels. Each reason label is binary (0 or 1), and a trial may have more than one reason label equal to 1. Other includes other or unspecified stopping reasons. This is common in the real world, where data acquisition devices/sensors fail, or human annotators are too expensive/inaccessible. This project will therefore consist of two parts:
   (a) Part 1: Use at least one unsupervised learning method (optionally combined with supervised learning techniques) to automate the addition of labels to the data examples with missing labels (i.e., augment your dataset). Generate missing reason labels in `part1_train.csv`, select a model and combinations of features using validation performance on `part1_val.csv`, and predict the five reason labels for `part1_test.csv`. Discuss the features you choose and why some features fail to improve the model performance.
   (b) Part 2: Build a binary classifier to predict clinical-trial outcomes for each trial phase (Phase I, II, and III), analyze its prediction errors across reason categories using the reason labels from Part 1, and improve the classifier. Discuss the results even if your modification does not improve F1.

2. **Project Motivation** The project is intended to give you experience working on both unsupervised and supervised learning methods in a more realistic setting. It is also intended to give you experience working with some of the topics covered in the last five weeks of class, for which this project is the evaluation. This means, in this project, you should evaluate at least one of the unsupervised algorithms proposed in the grading rubric at the end of this document, or maybe even machine learning methods we haven't explicitly covered in class. Here is a list of things that you should **not** do in this project:
   (a) Please do not try to reverse engineer the labels on the test set.
   (b) Please do not try to find the data source online and train your model on the fully labeled dataset.
   Violations of the above restrictions will result in a severe penalty for this project.

3. **Python Packages You Should Use** You are allowed to use well-known libraries such as `scikit-learn`, `scikit-image`, `numpy`, `scipy`, and `rdkit` (for the SMILES column). When required, we only ask that you develop your algorithms in `PyTorch` instead of `Keras`, `TensorFlow`, etc. **The RDKit package.** RDKit is an open-source Python library for cheminformatics, i.e., for working with molecules on a computer. Install it with `pip install rdkit` (it also works on Google Colab). In this project, its main use is to turn the `drug_smiles` strings into numerical features:

   - **Parsing.** `Chem.MolFromSmiles` reads a SMILES string into a molecule object (it returns `None` if the string is invalid).

   - **Fingerprints.** A Morgan (circular) fingerprint is a fixed-length binary vector (e.g., 1024 bits) in which each bit indicates whether the molecule contains a particular small substructure.

Molecules with similar structures have similar fingerprints, so fingerprints can be used directly as features in `scikit-learn`.

- **Descriptors.** Summary properties such as molecular weight (`Descriptors.MolWt`) and lipophilicity (`Descriptors.MolLogP`).
- **Similarity.** The Tanimoto similarity between two fingerprints (between 0 and 1) measures how structurally similar two drugs are, and can be used as a similarity measure for clustering or nearest-neighbor methods.
- A trial may have several matched drugs (separated by semicolons in `drug_smiles`) or none, so you need to decide how to combine fingerprints across drugs and how to handle trials without a matched drug. The example below takes the element-wise maximum across drugs (a bit is 1 if any drug in the trial has that substructure) and uses all zeros for trials without a matched drug; other choices are possible.

```
import numpy as np
import pandas as pd
from rdkit import Chem, DataStructs
from rdkit.Chem import Descriptors, rdFingerprintGenerator
gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=1024)
mol = Chem.MolFromSmiles("CN(C)C(=N)NC(N)=N")      # metformin
fp = gen.GetFingerprintAsNumPy(mol)                 # 1024 bits; 17 are 1
print(Descriptors.MolWt(mol), Descriptors.MolLogP(mol))   # 129.17, -1.03
aspirin = Chem.MolFromSmiles("CC(=O)Oc1ccccc1C(=O)O")
print(DataStructs.TanimotoSimilarity(gen.GetFingerprint(mol),
                                     gen.GetFingerprint(aspirin)))  # 0.05
def trial_fingerprint(smiles_field):
    """Combine the fingerprints of all matched drugs in one trial."""
    if pd.isna(smiles_field):
        return np.zeros(1024)                       # no matched drug
    fps = [gen.GetFingerprintAsNumPy(m)
           for s in smiles_field.split(";")
           if (m := Chem.MolFromSmiles(s)) is not None]
    return np.max(fps, axis=0) if fps else np.zeros(1024)
train = pd.read_csv("part1_train.csv")
X_fp = np.vstack(train["drug_smiles"].apply(trial_fingerprint))  # (1171, 1024)
```

4. **Dataset** The dataset is available on Kaggle and on Canvas under Week 8, and is based on clinical-trial data from the HINT benchmark (Fu et al., *Patterns*, 2022) and ClinicalTrials.gov.

   The datasets are provided separately through the two Kaggle competitions.

   The Part 1 competition provides four CSV files: `part1_train.csv`, `part1_val.csv`, `part1_test.csv`, and `pathway_catalog.csv`. The Part 2 competition provides three CSV files: `outcome_train.csv`, `outcome_val.csv`, and `outcome_test.csv`.

   Trial identifiers. Trials are identified by anonymized IDs (e.g., `TR00042`); NCT numbers are not provided and have been removed from all text fields. The same trial has the same ID in both parts: the `ID` column in the Part 1 files matches the `trial_id` column in the Part 2 files. Use these IDs

only for identifying and matching trials, not as a predictive feature.

Part 1 data. The Part 1 files contain 1,561 trials, from all phases, that stopped early (terminated, withdrawn, or suspended) and have a recorded reason for stopping. For most trials the reason comes from the `why_stop` field of ClinicalTrials.gov; for 42 trials without a recorded reason, the reason was recovered from other parts of the ClinicalTrials.gov record (e.g., the results section) and labeled by the course staff. Trials are split by start year: `part1_test.csv` contains the 312 latest trials (starting 2010–2016), and the earlier trials are split at random into `part1_train.csv` (1,171 trials) and `part1_val.csv` (78 trials).

Each file contains `ID`, `start_year`, 150-dimensional reason embeddings (`emb_0` through `emb_149`), five binary reason-label columns (Safety, Funding, Efficacy, Recruitment, and Other; they are not provided in `part1_test.csv`), the trial's eligibility `criteria`, `diseases_cleaned`, `drugs`, `icdcodes`, and drug-knowledge features. The reason embeddings were computed from the stopping-reason text with a pretrained sentence-embedding model, reduced to 150 dimensions with PCA, and randomly rotated, so individual dimensions have no meaning; the embedding values are small, so standardize them before use.

The drug-knowledge features link each drug in the `drugs` column to its molecular structure and to the biological pathways it may affect. Drugs were matched by exact name to a DrugBank-derived table of small-molecule drugs, so biologics (e.g., antibodies) and drugs listed under other names are not matched; about 56% of Part 1 trials have at least one matched drug, and the drug columns are empty for the rest. Each column lists values for all matched drugs of a trial, separated by semicolons:

- **Matched drugs.** `matched_drugs` lists the drug names that were matched.
- **Drug pathways.** A drug does not act on a pathway directly: it binds one or more protein *targets* (`drug_targets`, given as gene symbols), and each target participates in pathways curated by the Reactome database (`drug_pathways`, given as Reactome IDs; `pathway_catalog.csv` maps each `reactome_id` to its `pathway_name`, `pathway_level`, and the number of drugs and targets in it). A pathway is a set of molecular reactions that together carry out a biological function, such as "Hemostasis" (blood clotting) or "Signaling by GPCR." A drug's pathways are therefore the union of the pathways of all its targets. The column `drug_actions` describes how the drug affects its targets (e.g., inhibitor, agonist); actions are pooled across all targets of the trial's matched drugs, so the link between a specific action and a specific target is not available.
- **Pathway level.** Reactome organizes pathways as a hierarchy. `pathway_level` (in `pathway_catalog.csv`) is a pathway's depth in that hierarchy: level 0 contains the 29 broadest categories (e.g., "Signal Transduction," "Metabolism," "Immune System"), and higher levels are increasingly specific sub-pathways (the deepest is level 9). For example, "Formation of Fibrin Clot" (level 1) is part of "Hemostasis" (level 0) and contains "Intrinsic Pathway of Fibrin Clot Formation" (level 2). A target that belongs to a specific pathway is also listed under every broader pathway that contains it. `pathway_level` describes the pathway, not the drug, and is an ordinal depth rather than a measured quantity.
- **SMILES.** The `drug_smiles` column encodes each matched drug's chemical structure as a

text string (Simplified Molecular Input Line Entry System). Letters are atoms (`C` carbon, `N` nitrogen, `O` oxygen), `=` and `#` are double and triple bonds, parentheses are branches, matching digits close rings, `@`/`@@` mark stereochemistry, and `.` separates disconnected parts (e.g., a salt). For example, metformin is `CN(C)C(=N)NC(N)=N`. The same molecule can be written as several different valid SMILES strings. Libraries such as RDKit convert SMILES into numerical representations (molecular descriptors or fingerprints).

The original why_stop text is not provided. You may only use a subset of available features to maximize the model performance.

In `part1_train.csv`, 157 trials have visible reason labels (0 or 1): the earliest 10% of trials by `start_year` (117 trials), plus 40 additional trials that were selected to include more examples of the rarer reasons (Safety, Funding, Efficacy, and Other). The remaining 1,014 trials have all five reason labels masked as -100. All reason labels are visible in `part1_val.csv` and hidden in `part1_test.csv`. Because the labeled trials are mostly early and the test trials are the latest, the frequency of each reason differs between the labeled, unlabeled, and test trials (for example, Recruitment is more common in early trials and Safety in later ones); consider this when designing your method.

Evaluate Part 1 using sample-wise F1 (F1 averaged across trials), and report F1 for each reason label and the macro-averaged F1 (F1 averaged across the five labels). Select your model using `part1_val.csv` and submit the five predicted reason labels for each trial in `part1_test.csv` via Kaggle: one row per trial with columns `ID` and `Predicted`, where `Predicted` lists the predicted reasons separated by spaces (e.g., `Recruitment Other`). Discuss the features you choose and report why including everything might fail to improve the performance of the model.

Part 2 data. The outcome files contain clinical trials from all three phases, with binary outcome labels indicating whether each trial met its primary endpoint (label=1) or not (label=0). Each row is one trial in one phase: a trial in a combined-phase study (e.g., Phase I/II) appears once for each phase, with the same label. Success means different things in different phases, so train a separate model for each phase, or one model that uses the phase. The files include `ID` (the trial ID followed by the phase, e.g., `TR00042_II`), `trial_id`, `phase` (I, II, or III), `phase_detail` (e.g., "phase 1/phase 2"), `criteria`, `diseases_cleaned`, `drugs`, `icdcodes`, the drug-knowledge columns described in Part 1, `start_year`, and the binary outcome label (not in `outcome_test.csv`). The labels come from the HINT benchmark; we corrected 7 labels that contradicted the trial's ClinicalTrials.gov record, but some label noise likely remains.

`outcome_train.csv` (8,037 rows) and `outcome_val.csv` (1,013 rows) contain trials that started up to 2013, split at random by trial, so the same trial never appears in both. `outcome_test.csv` contains 1,500 rows (500 per phase) from trials that started later (2013–2022); none of them appear in any other file, including the Part 1 files. Use `outcome_train.csv` to train your classifiers and `outcome_val.csv` for model selection and evaluation, then submit the predicted probability of label=1 for each row of `outcome_test.csv` via Kaggle (columns `ID` and `label`). The Kaggle score is the log loss, which, because each phase has the same number of test rows, equals the average of the per-phase log losses. Expect lower performance on the test set than on the validation set, since the test trials are from later years; discuss this gap in your report. You may consider start_year as an additional predictive feature. Report validation F1, ROC-AUC, and PR-AUC (average precision) for each phase, treating label=1 as the positive class. Compute F1 from binary predictions and

ROC-AUC and PR-AUC from predicted probabilities for label=1.

For unsuccessful trials (label=0) in `outcome_val.csv`, match `trial_id` to the Part 1 `ID` to obtain their available or predicted reason labels from Part 1. Many unsuccessful trials ran to completion without stopping early and therefore do not appear in Part 1; treat them as a separate "no recorded stopping reason" group. A trial can have more than one reason, so it can count toward several reason categories. Analyze each phase separately, so that a trial appearing in two phases is not counted twice within a phase, and state which reason labels are true labels and which are your Part 1 predictions.

For each reason category (including the no-reason group) and each phase, report the number of trials and the fraction incorrectly predicted as successful (label=1).

Given the analysis above, improve the binary classifier, and compare its overall performance (F1, ROC-AUC, and PR-AUC) and error rates across reason categories with the original model.

Reason embeddings and reason labels should be used only for error analysis, not as predictive inputs to the binary classifier.

PROJECT GRADING

We will now outline the criterion by which we will grade the project. Note, some projects will be more creative than others; some projects will achieve higher performance than others. We will provide room for extraordinary work in one category to compensate for deficiencies in another category. These are the general areas we will look into. Concretely, the midterm project will be graded on a scale of 100 points, but each section is assigned points so that the sum total can exceed 100 points. Your final project score will be capped at 100 points. You should aim to do a good job in all areas.

1. **Creativity (35 points maximum)** How creative and/or diverse is the approach? Have you implemented / tried various algorithms? Are multiple classes of algorithms compared? An example of a project that we would count as creative is implementing and comparing at least two options for each of the below categories (2 reason-labeling methods, 2 binary classifiers or settings, and 2 feature/preprocessing choices). These counts are examples, not additional minimum requirements. Alternatively, you can create *ensemble methods* that combine the predictions or decisions of multiple individual models. Here are some suggestions (you may use other suitable methods as well):

   (a) Distribution based clustering methods (e.g., GMM)
   (b) Density based clustering methods (e.g., DBSCAN, note that we have not covered DBSCAN in the class but you can try it out)
   (c) Centroid-based clustering methods (e.g., K-Means)

   Some combination of the following supervised methods:

   (a) Multinomial Naive Bayes (Part 2, with nonnegative text features)
   (b) One-vs-rest Logistic Regression (Part 1) or binary Logistic Regression (Part 2)
   (c) K Nearest Neighbors (multi-label classification in Part 1; binary classification in Part 2)

   Possible text feature representations for Part 2:

(a) Bag of words
(b) N-gram
(c) TF-IDF features

To be fair to students who have not taken a ML class, please **do not** use deep learning methods in this midterm project. You will have the chance to do that in the final project.

2. **Insight (35 points maximum)** Does the project reveal some insight about the choice of hyperparameters, architectures, etc. on the performance of the algorithm? Is there reasonable insight / explanation / intuition into the results? (i.e., you should not just blindly apply different algorithms to a problem and compare them.) For Part 2, discuss differences in error rates across reason categories, considering category sizes and possible errors in predicted reason labels.

3. **Performance (30 points maximum)** Does the project achieve relatively good performance on the problem, given the computational resource that you have access to? How do different algorithms compare? Did you to optimize the algorithms chosen instead of just randomly training a few different algorithms without optimizing them? For Part 2, evaluate the modification and its trade-offs.

4. **Report Writeup (15 points maximum)** Are the approach, insight, and results clearly presented and explained? Dissemination of results is an important component to any project.

PROJECT REPORT WRITEUP

Each group should submit a writeup of their project work, exceeding no more than 5 pages including figures. References are excluded from the 5 pages (e.g., they may overflow onto a 6th page and be numerous). It is fine to be below the page limit; this is the maximum. We appreciate conciseness. We will also ask you to submit your **code** and **dataset**, so that we can validate your results.

The writeup must adhere to the following template, accessible via this link – so that we can judge all writeups in the same manner without having to worry about different font sizes, etc.

Your report writeup should contain the following sections:

1. **Abstract** A brief description of what you did in the project and the results observed.
2. **Methods** State the details about your models. This section should summarize all the methods that you have tested. Specifically, explain the details about 1) your model (e.g., how did you initialize the cluster centroids or GMM clusters) and 2) the training (e.g. for EM algorithm, the threshold parameter determined to stop the algorithm convergence, the number of iterations, etc.).
3. **Results** State the results of your experiments. This section should include a table summarizing the performance of your models.
4. **Discussion** Discuss insights gained from your project, e.g., what resulted in good performance, and any hypotheses for why this might be the case.
5. **References** List references used in your writeup. We expect you to cite all papers, books and websites used for ideas, code and phrasing. Please review the Canvas course syllabus for our policy on Academic Integrity.